

To type out a Claude custom skill, you must use **exact uppercase formatting for the SKILL.md file name**, start with a YAML frontmatter block bound by triple dashes (---), and follow it directly with standard Markdown instructions. [1, 2]

The exact file syntax, variable rules, and structural layout are detailed below. [3]

## Complete SKILL.md Syntax Blueprint

```
---
name: pull-request-helper
description: Generates clean, well-formatted PR descriptions. Use when the user asks to
write a pull request description, create a PR, or summarize branch changes.
version: 1.0.0
allowed-tools: read, grep, execute
---
```

### # Pull Request Helper

#### ## Overview

This skill automates the creation of a clear pull request description.

#### ## Instructions

1. Analyze the uncommitted or recently committed changes in the workspace.
2. Identify the core components of the changes, focusing on logic updates.
3. Replace placeholders with dynamic inputs using the context from your request.

#### ## Arguments and Live Context

\* Main Argument: Use ``$ARGUMENTS`` to fetch the ticket number or extra developer context.

\* Active Branch Name: ``! git branch --show-current``

#### ## Output Format

Structure your response exactly using these three headers:

### 🚀 Summary

### 🛠️ Changes Made

### 🔍 Verification Steps

---

## Breakdown of Key Syntax Components

### 1. The YAML Frontmatter (Required)

The metadata block controls how and when Claude activates your skill. It must sit at the

absolute top of the file: [1, 2, 4]

- 
- name: Must be strictly lowercase, kebab-case only, and match your containing folder's name exactly. This becomes your explicit manual slash command (e.g., /pull-request-helper). [3, 5, 6]
- description: The most critical field. You must describe **what it does** and explicitly state **trigger phrases** (e.g., "Use when the user asks..."). Claude scans this metadata to auto-trigger the skill during normal chat. [5, 6, 7]
- allowed-tools: (Optional) Restricts or explicitly allows specific tool permissions when this skill executes. [8, 9]
- 

## 2. Live Context and Variable Syntax

You can inject dynamic inputs directly into the body text using two distinct syntaxes: [10]

- 
- **Dynamic Arguments:** Use \$ARGUMENTS (or shorthand \$0, \$1). If you type /pull-request-helper API-404, Claude will automatically swap \$ARGUMENTS out for API-404 before processing.
- **Live Shell Context:** Use the **! command** syntax enclosed in backticks to query live workspace info. When Claude reads your file, it executes the shell command immediately and inserts the raw terminal output directly into the prompt text before evaluating it. [10, 11]
- 

## 3. Markdown Body Instructions

Beneath the frontmatter, write the rest of the file like an onboarding manual. Keep rules structured with Markdown headers (#, ##) and numbered lists so Claude treats them as high-priority, sequential steps. [3, 12, 13]

For a complete step-by-step video tutorial demonstrating how to type, save, and test this file structure locally:

If you want, let me know:

- 
- What specific **workflow or task** are you trying to automate?
- Do you need help formatting a **live shell command (!)** for your workspace?
- 

[1] <https://www.youtube.com>

[2] <https://codemeetai.substack.com>

[3] <https://www.skillsdirectory.com>

[4] <https://www.agensi.io>

[5] <https://resources.anthropic.com>

- [6] <https://code.claude.com>
- [7] <https://www.mdskills.ai>
- [8] <https://agentfactory.panaversity.org>
- [9] <https://www.reddit.com>
- [10] <https://sherlock.xyz>
- [11] <https://www.damiandemasi.com>
- [12] <https://resources.anthropic.com>
- [13] <https://bibek-poudel.medium.com>